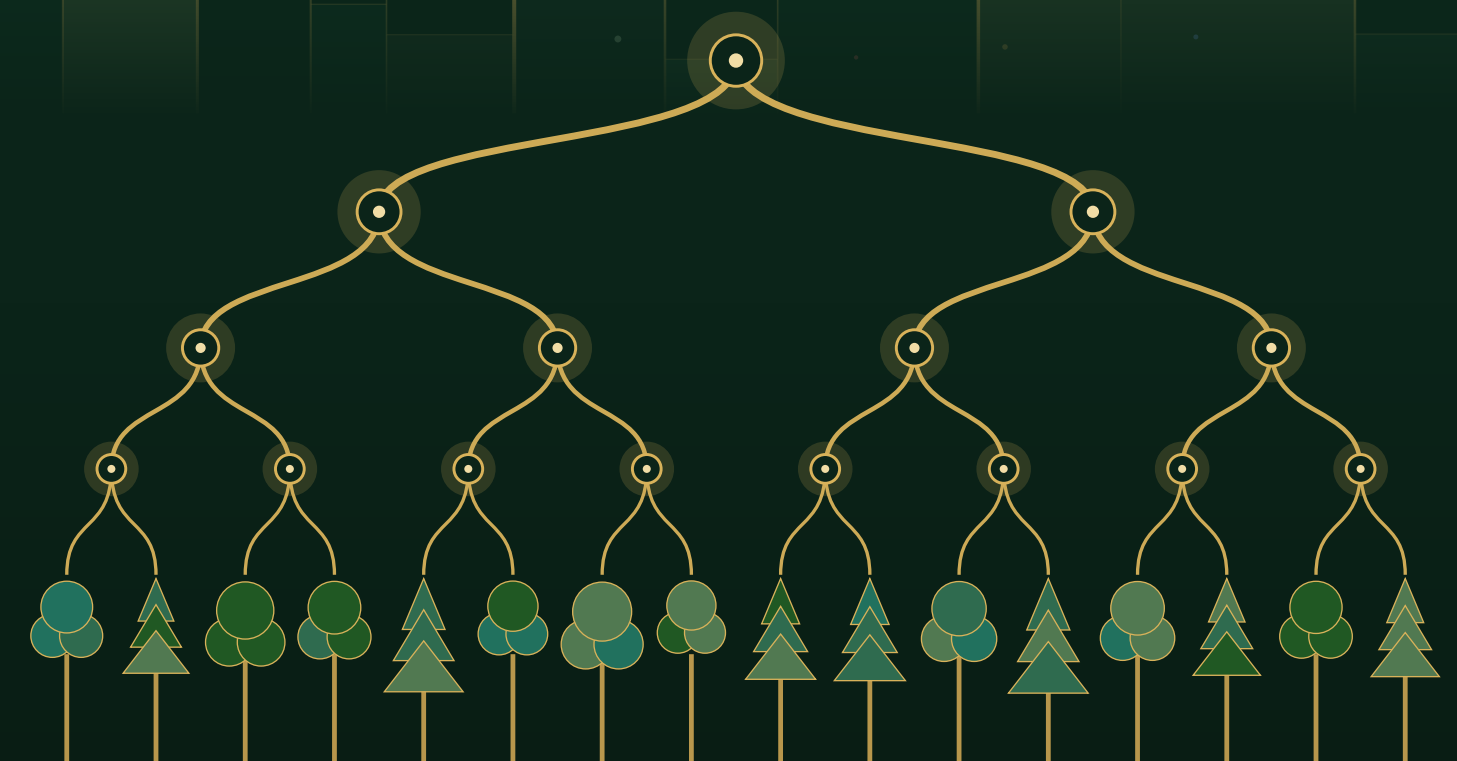


A HANDWRITTEN THEORY, FROM THE ROOT TO THE LEAVES

THE ENCHIRIDION

for Tree Based Algorithms Theory

CLASSIFICATION & REGRESSION TREES · PRUNING · RANDOM FOREST · EXTRA TREES
ADABOOST · GRADIENT BOOSTING · XGBOOST · CATBOOST · LIGHTGBM



REDA LOURIKI

enchiridion (n.), from the Greek: "that which is held in the hand"; a handbook.

Preface

THIS book started as handwritten notes. During the summer of 2026, chapter after chapter, I wrote by hand the whole theory of tree-based algorithms, the way I would have liked someone to explain it to me: starting from a single question asked to the data, and ending with the algorithms that win machine learning competitions today.

An *enchiridion* is, in Greek, “something that is held in the hand”: a handbook. The name fits twice. It is a small book you can keep close to you, and it was written, literally, by hand.

Who made this book

Author. The content of this book (the explanations, the examples, the demonstrations and the way of telling them) was created by **Reda Louriki**, in handwritten notes.

Typesetting. The handwritten notes were transcribed and turned into L^AT_EX by an AI, **Claude Opus 5.5** (Anthropic). The author’s wording and examples were kept; only grammatical errors and mathematical errors were corrected. The part of the Gradient Boosting chapter that the notes left unfinished was completed in the same spirit.

Every time a correction changes the meaning of what was written in the notes (a number, a formula, a name, a date), it is signalled with a footnote marked **Editor’s note**, so you can always see what the notes said and why it was changed. The list of all these changes, and of the completed parts, is gathered at the end of the book, in *Notes on this Edition*.

How to read this book

The notes were written with several colours of ink, and each colour had a job. The book keeps the same code:

Red	the titles of the parts, and the warnings ⚠
# Step:	the steps of each algorithm
Blue	the questions you should be asking yourself at that moment
Green	“Algorithm:” headings, remarks and important consequences
Gold boxes	the sentences worth remembering

The chapters follow the historical and logical order of the ideas. We begin with one tree (classification, regression, pruning), then we let many trees vote (Random Forest, Extra Trees), and finally we let the trees learn from each other’s errors (AdaBoost, Gradient Boosting), until

the modern gradient boosted decision trees: XGBoost, CatBoost and LightGBM. Each chapter uses what the previous ones built, so reading them in order is the best way.

No advanced prerequisite is needed: some probability (independence, variance, covariance), derivatives, and the curiosity to ask “why?” at every line.

Reda Louriki
2026

Contents

Preface	i
I Decision Trees: Classification Trees	1
I.1 Gini Impurity	3
I.2 The Algorithm	4
I.3 Good to Know Info	7
II Decision Trees: Regression Trees	9
II.1 But How About Multiple Features?	12
III Pruning: Cost Complexity Pruning	13
IV Random Forest	17
IV.1 Advantages of Random Forest	19
IV.2 The Math Behind Random Forest	19
V Extra Trees	23
VI AdaBoost	25
VI.1 Making a Prediction	28
VII Gradient Boosting	29
VII.1 Mathematical Details	32
VII.2 Why “Gradient”?	34
VII.3 Gradient Boosting for Classification	35
VII.4 Good to Know Info	37
VIII GBDT Based Algorithms: XGBoost	39
VIII.1 XGBoost	39
IX GBDT Based Algorithms: CatBoost	43
IX.1 Target Encoding	43
IX.2 Ordered Target Encoding	44
IX.3 The Training	45
IX.4 Oblivious Trees	47

X	GBDT Based Algorithms: LightGBM	49
X.1	Histograms	49
X.2	EFB	50
X.3	GOSS	51
X.4	Leaf-Wise Growth	52
	Notes on this Edition	53

Decision Trees: Classification Trees

WE are now starting our machine learning journey, and what's better to start with than decision trees?

A decision tree is an algorithm whose main goal is to reduce the “impurity” of the data by asking questions and taking decisions.

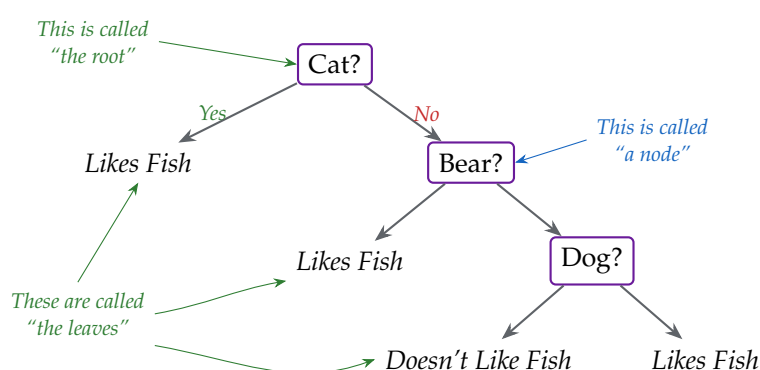
In our routine, we often think in the same way as a decision tree: *If thirsty, and I don't want water, then buy juice!* And it seems pretty effective, since our lives rely on these decisions.

What does it look like?

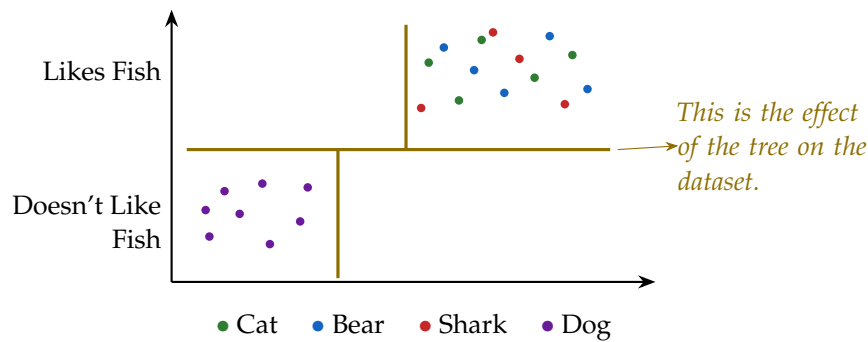
Imagine we have this dataset:

Animals	Likes eating fish
Cats	Yes
Cats	Yes
Dogs	No
Sharks	Yes
Bears	Yes

The tree would look like this:



*“Classification and Regression Trees”
This is the “CART” methodology, and the most common one. It has only two choices: Left: True, Right: False.*



Algorithm:

The first question we could have in mind is:

“How do we choose the root? Why is it cat, and not dog or bear...?”

Well, there are many ways to answer this question, but as I said in the beginning: the main idea is to reduce the “impurity” of the data, so we choose the split that reduces it the most.

What’s impurity in data?

Data is *impure* when it has multiple classes, and if we choose a random sample from the data, we won’t be 100% sure which one it is.

For example: in the preceding data, if we randomly choose an animal, we won’t know if it likes fish or not, because the data has a mixture of *Yes* and *No*. But if we choose “cat”, we are sure that every cat that we have chosen would like fish, so the data now becomes “pure”.

“By choosing cat as a split, we reduced impurity.”

How could we measure the impurity reduction of a split?

There are three main ways:

- Information gain
- Entropy
- Gini impurity

We adopt the CART algorithm, so it’s preferable to choose the Gini impurity. Notice that information gain and entropy would, in practice, lead to (almost always) the same result.¹

Remark: CART was introduced in 1984;² at that time, computational power was limited, so Gini impurity was introduced with CART since it was the fastest method at that time. Nowadays computers are super fast, so we don’t see the difference.

¹ **Editor’s note.** The handwritten notes say they “would lead to the same result”. Gini and entropy produce the same split in the vast majority of cases, but not in every case, so “almost always” was added. Also note that information gain is not a separate impurity measure: it is the *reduction of entropy* produced by a split (see the end of this chapter).

² **Editor’s note.** The notes say 1980. *Classification and Regression Trees* by Breiman, Friedman, Olshen and Stone was published in 1984.

1.1 Gini Impurity

As I said before, there are multiple methods to measure the “impurity”. One of them, and the most famous one, is *Gini impurity*, named after the Italian statistician *Corrado Gini*.³ Its formula is:

$$\text{Gini_impurity} = 1 - \sum_i p_i^2$$

where p_i is the probability of having each class, squared.

Why and how does this formula help us to measure mathematically the impurity (mixture) in data?

Let’s dive deeper into the formula together.

Property

- $P(A \cap B) = P(A) \times P(B)$ iff A and B are independent.
- $P(A \cap A) = P(A)^2$ iff $P(A)$ is either 0 or 1.

Demo:

Suppose $P(A \cap A) = P(A)^2$ (i.e. A is independent of itself).⁴ Since $A \cap A = A$:

$$\begin{aligned} P(A \cap A) = P(A)^2 &\iff P(A) = P(A)^2 \\ &\iff P(A) - P(A)^2 = 0 \\ &\iff P(A)(1 - P(A)) = 0 \\ &\iff \boxed{P(A) = 0 \text{ or } P(A) = 1} \end{aligned}$$

Now we can say that:

- * In the Gini impurity, p_i^2 means $P(i, i)$: the probability of picking a sample of class i and then *again* a sample of class i , i.e. of picking the same class successively. The two picks are made with replacement, so they are two *independent* events and, by the first property, $P(i, i) = p_i \times p_i = p_i^2$.⁵
- * $\sum_i p_i^2$ is the sum of all probabilities of picking the same class successively.
- * For example:

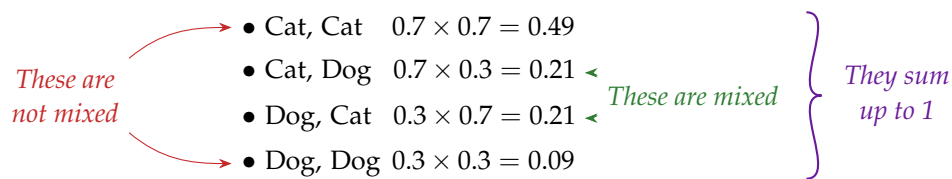
Classes	Proba of having a class
Cat	0.7
Dog	0.3

³**Editor’s note.** The notes say “introduced by” Corrado Gini. Gini introduced the Gini coefficient/index (1912); the impurity measure used by CART is named after his index.

⁴**Editor’s note.** The notes wrote “since A is independent of itself” as if it were always true. It is the *hypothesis* of the demonstration: the result shows that an event is independent of itself only when $P(A) \in \{0, 1\}$.

⁵**Editor’s note.** The notes wrote $p_i^2 = P(i \cap i)$ using the second property. By that very property, $P(A \cap A)$ equals $P(A)$, not $P(A)^2$, for a single event. The square comes from two *independent* draws (first property), so the sentence was rewritten.

Then we have 4 possibilities:



Big idea: Gini impurity measures the mixture of the data.

We can already compute the probability of the “not mixed data” (p_i^2), and since all the possibilities sum up to 1, then:

This is where it comes from

$$\underbrace{P(\text{Mixed Data})}_{\text{this is the Gini impurity}} = 1 - \sum_i p_i^2$$

With the table above: $1 - (0.49 + 0.09) = 0.42 = 0.21 + 0.21$, exactly the probability of the two mixed possibilities.

Now we know the Gini impurity and how to compute it, we then need the algorithm.

1.2 The Algorithm

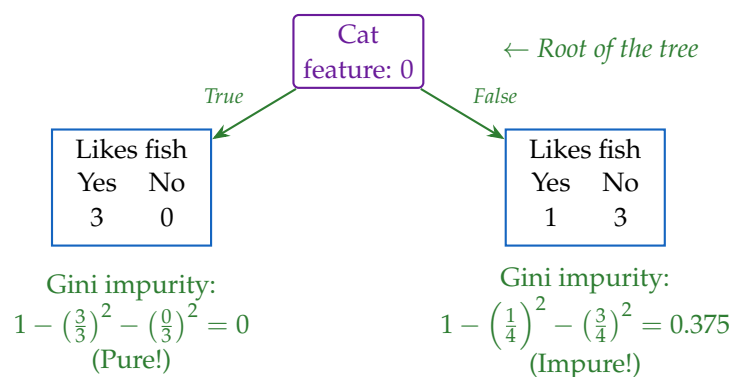
#Step 1:

Collect and arrange the data:

$$X = [[\text{feature}_1], [\text{feature}_2], \dots, [\text{feature}_n]], \quad Y = [\text{prediction}]$$

#Step 2:

From the data, we try to find the best split, by seeing which class (i.e. which value of a feature, like “cat”) from all the features reduces impurity the most. By using the Gini impurity, we can determine the root (or node, next split) of our tree. Let’s say “cat” is the class that reduces impurity the most:



△ Weighted Gini Impurity

We will need it later: it's the impurity of the node "cat". To compute it:

$$G_{\text{cat}} = \frac{n_T}{N} G_T + \frac{n_F}{N} G_F$$

Diagram illustrating the calculation of Gini impurity for the "cat" node:

- G_{cat} : Gini impurity of cat
- $\frac{n_T}{N}$: number of samples if true (3)
- G_T : Gini impurity if true (0)
- $\frac{n_F}{N}$: number of samples if false (4)
- G_F : Gini impurity if false (0.375)
- $N = n_T + n_F = 7$: Total number of samples covered by the node

In our case:

$$G_{\text{cat}} = \frac{3}{7} \times 0 + \frac{4}{7} \times 0.375 \approx 0.214 \quad \leftarrow \text{"Gini impurity of node cat"}$$

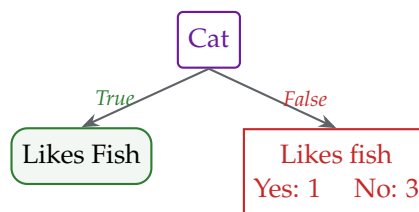
Step 3:

We now need to decide whether we need to split the leaves or not. The main criterion is of course "impurity".

Condition n° 1: Stop if the leaf is pure (Gini impurity = 0).

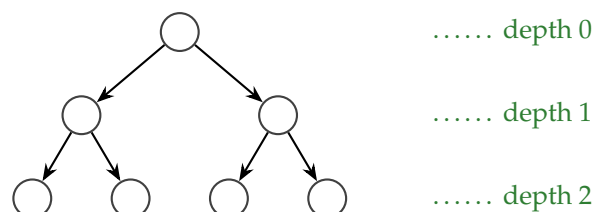
In our case, that corresponds to the left child of our node "cat". As computed, the left leaf is pure.

We then turn the pure leaf into a result: since there are more "Yes" than "No" (3 against 0), we turn the leaf into Likes Fish. And the tree becomes:



Condition n° 2: Max depth reached.

Definition of depth:



To avoid overfitting (we will talk about this in detail), frameworks like scikit-learn allow defining a maximum depth for the tree. If we set max depth to 1, the tree will be truncated and we will not split any other leaf.

Condition n° 3: Min samples required.

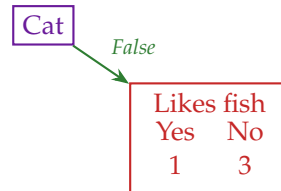
Easy to understand, it's simply: "Don't split a node if we have less than, for example, 2 samples."

"If none of these conditions are verified, we split the node."

#Step 4:

Before "re-splitting" we need to do something.

Since this node is the result if cat is False, then to compute the Gini impurity we need a new dataset.



If at the beginning the dataset was:

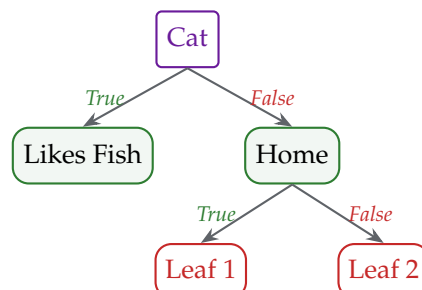
Animals	Habitat	Whether they like fish or not
Dog	Home	No
Cat	Home	Yes
Cat	Home	Yes
Tiger	Jungle	No
⋮	⋮	⋮

Since `is_cat` is *false*, we will consider only the new data that doesn't contain cat:

Animals	Habitat	Whether they like fish or not
Dog	Home	No
Tiger	Jungle	No
Bear	Forest	Yes
Dog	Home	No

(Note that if `is_cat` was true, then we would consider only samples with cat in it.)

Now, after computing the Gini impurity and finding the best class (let's say "Home"), we can then represent:



⚠ Important remark!

Remember that we computed the weighted Gini impurity of a node; this will help us to know if a tree is correct or not. Let me explain:

Before, we said that cat's node impurity is "0.214".
Let's say "Home" impurity is 0.392.

⚠ "This should never happen!" ⚠

From a node to its child, the node impurity (weighted Gini impurity) should decrease, remember that.⁶

#Step 5:

Repeat! Until one stoppage condition is satisfied.

*The decision tree algorithm is recursive
(see my implementation from scratch on my GitHub).*

1.3 Good to Know Info

1/ As I said before, the method of tree making that we have treated is the "CART" methodology, and it's the one used by scikit-learn. But there are others:

ID3: "Iterative Dichotomiser 3"

- Used for classification only.
- Split criterion: information gain.
- Characteristics: multiway splitting (not only true or false). No pruning.
- Weakness: can't handle continuous variables.

C4.5 / C5.0: successors of ID3. They solve some ID3 problems by introducing the *gain ratio*.

And there is much more.

2/ Impurity measure methods:

- **Gini impurity.**
- **Entropy:** computes how much "chaos" there is in the data:

$$H = - \sum_i p_i \ln(p_i)$$

- **Information gain:** used by ID3, computes the difference of entropies between the parent and the children:

$$\text{Information gain} = H(\text{parent}) - \sum_k \frac{n_k}{N} H(\text{child}_k)$$

⁶**Editor's note.** More precisely: the weighted Gini impurity of the children of a split can never be larger than the Gini impurity of the node being split (the Gini function is concave). With the data above, "Home" gives $\frac{2}{7} \times 0 + \frac{2}{7} \times 0.5 = 0.25 \leq 0.375$; and measured on the whole tree (all 7 samples), the impurity goes from 0.214 to $\frac{2}{7} \times 0 + \frac{4}{7} \times 0.25 \approx 0.143$. A value like $0.392 > 0.375$ would signal a bug.

where the entropies of the children are weighted by their number of samples, exactly like the weighted Gini impurity.⁷

⁷**Editor's note.** The notes wrote $H(\text{parent}) - H(\text{child})$. When a node has several children, their entropies must be weighted by their share of the samples.

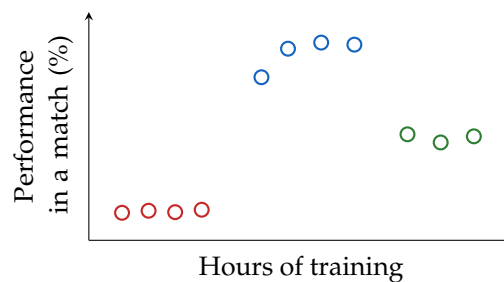
Decision Trees: Regression Trees

WE have discussed the classification part of the decision tree algorithm; now let's talk about "Regression".

A "Regression Model" is a model that predicts a numerical value rather than classes, trained on numerical continuous observations.

As said in the introduction, a *regression tree* would predict numerical values and would be trained on numerical data, rather than predicting classes and being trained on classes.

This is the idea:



(At the time I am writing this chapter, Spain have just won the World Cup against Argentina.)

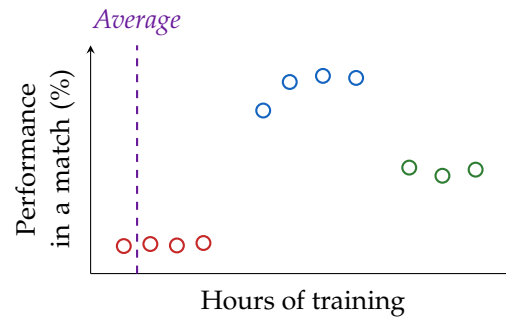
Our goal would be to separate the data into clusters by using a decision tree.

Algorithm:

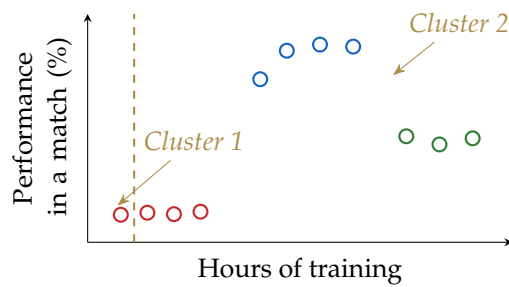
#Step 1:

Choose two (neighbouring) data points and average them (we average their *hours of training*).¹

¹**Editor's note.** The split threshold is the average of the *feature* values (x) of two neighbouring points; the target values (y) are only used afterwards, to compute the averages of each cluster.

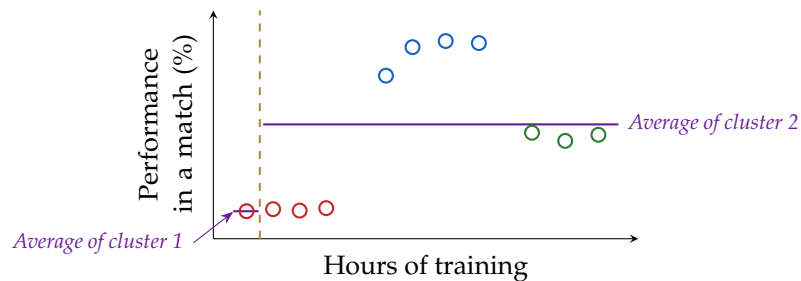


Then we separate the data initially into two clusters:

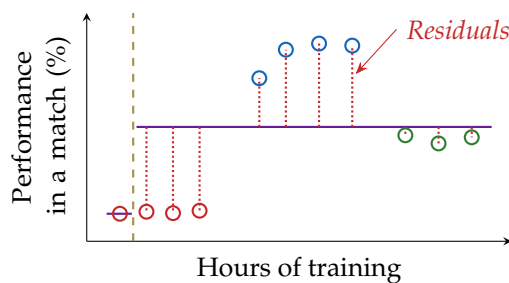


#Step 2:

We compute the average in each cluster.



We then compute the **sum of squared residuals**:

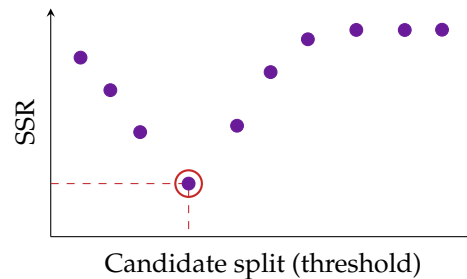


$$SSR = \sum_i (\text{Average} - y_i)^2$$

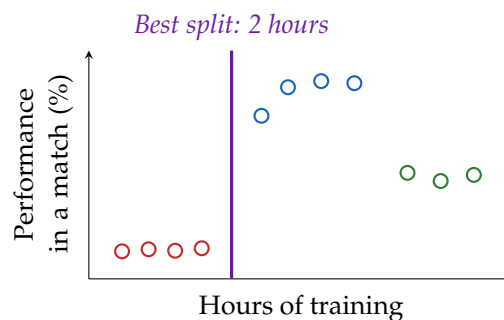
where y_i is the observed value (here the performance) of the sample i , and “Average” is the average of the cluster it belongs to.²

#Step 3:

We repeat the algorithm: we then try to find which “separation” did the best job minimizing the SSR.

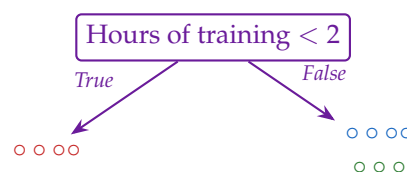


This split did the best job minimizing the SSR, which corresponds to this one:



We have now split the data into two clusters.

And the split would become the root node:³



#Step 4:

We repeat the same process for each cluster individually to get even more clusters.

⚠ Important Note:

To avoid overfitting, we set a condition to stop splitting into more clusters.

Condition n° 4: min_samples for splitting.

²**Editor's note.** The notes wrote X_i . The residuals are measured on the target y_i (the vertical axis), not on the feature.

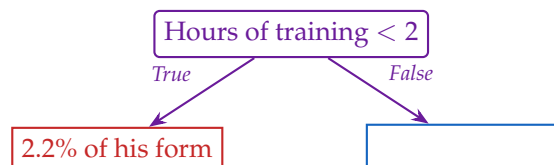
³**Editor's note.** The notes label this split “< 2 days” while the axis is *hours* of training; the unit was harmonised to hours.

Don't split the cluster if it contains less than `min_samples` (typically 20). In our case, let's take it 5.

So the first cluster (4 samples) wouldn't be divided anymore. It will become a leaf.

The leaf result is the average of all samples in the cluster.

Let's say that the average for the first cluster is 2.2%, then:

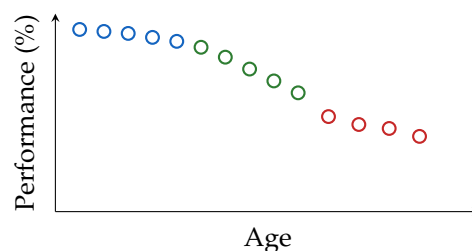


Since the node in blue contains more than 5 samples (7), we will reuse the algorithm in it!

And that's all for Regression Trees.

II.1 But How About Multiple Features?

Same as classification trees. Let's say we have:



We apply steps 1 and 2 and get the best split for each feature, and the split's SSR:

Age: SSR = 20.2
Hours of training: SSR = 19.7
Sex: SSR = 22.8

← This is the best split between all of them.

That's all! The feature (and threshold) with the lowest SSR becomes the node.

We can finally say that we have concluded with Decision Trees. Only one concept remains: "Cost Complexity Pruning". We will discuss it in the next chapter.

Pruning: Cost Complexity Pruning

ONE of the main problems that faces decision trees is *overfitting*, and it's caused by trees that are too deep: they fit the training data so well, but they don't do a pretty good job on testing data. Even though we have added some conditions to restrict the depth and unnecessary splitting, these conditions feel hard coded.

So that's why we introduce *pruning*.

In English, *pruning* means trimming by cutting away dead or "overgrown" branches.

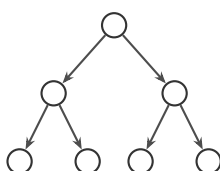
In machine learning, it's the same thing, but for decision trees, and the overgrown branches are the unnecessary splits that only fit the training data too much.

So, how does it work?

Algorithm:

#Step 1:

Take the full data (Train + Test)¹ and train a *big* tree on it. Let's say this is the tree:



#Step 2:

We introduce this objective function that we want to minimize (at first it was the impurity):

$$\mathcal{L} = I + \alpha |T|$$

The new objective function $\mathcal{L}$
 The impurity or error (Gini / SSR / Entropy) I
 Coefficient (we talk about it now) α
 Number of leaves in a tree $|T|$

¹**Editor's note.** "Test" here means the data used to choose α (it will be cut into folds in Step 4). If you want an honest final score, keep a separate hold-out test set that is never used during pruning.

This is a *regularization* method! The idea of this formula is to penalize the algorithm if it tends to add more branches and leaves. And the coefficient “ α ” determines by how much we would penalize the algorithm for adding leaves.

That’s why it’s called “Cost Complexity Pruning”.

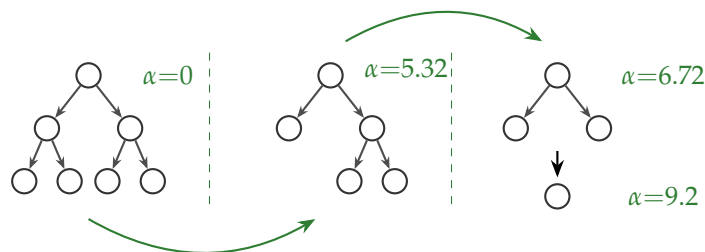
The whole pruning algorithm is to determine α !

#Step 3:

α determines “by how much we should penalize the algorithm for complexity”. That directly makes the algorithm “drop” some branches and makes the tree lighter. So α leads to dropping branches.

Strategy:

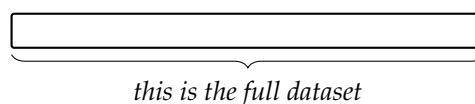
To determine α , we increase its value from 0 (no penalty) until one subtree drops:²



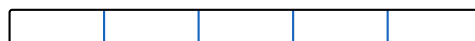
Now we have a list of trees and alpha values, until the tree became a leaf. Let’s call them $\alpha_1 = 0$, $\alpha_2 = 5.32$, $\alpha_3 = 6.72$ and $\alpha_4 = 9.2$. We will take the alpha values for the next step, and keep in mind these trees that we have made.

#Step 4: K-Fold Cross Validation

If these terms feel new to you, no worries. First, we take the whole dataset (Train + Test):



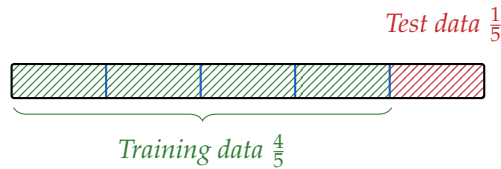
Then, we fold it into K folds (typically 5–10 folds):



We then use a cross validation method where we “leave one fold out”.³ It’s no big deal! The meaning is that if we fold the dataset into K folds, then $\frac{K-1}{K}$ of the data goes to the training data, and $\frac{1}{K}$ (leave one only) goes to the test data.

²**Editor’s note.** In practice, the next value of α does not need to be searched blindly: for every internal node t , the subtree T_t below it drops as soon as α reaches $\frac{I(t) - I(T_t)}{|T_t| - 1}$ (“weakest-link pruning”). The smallest of these values is the next α .

³**Editor’s note.** The notes call it “Leave One Out Cross Validation (LOOCV)”. That name is usually reserved for the special case $K = n$ (one sample per fold). What is described here, leaving one *fold* out each time, is K -fold cross validation.



Okay, but why choose the last fold as test data, and not the first or the second...?

Good question, let's go to Step 5 to answer it.

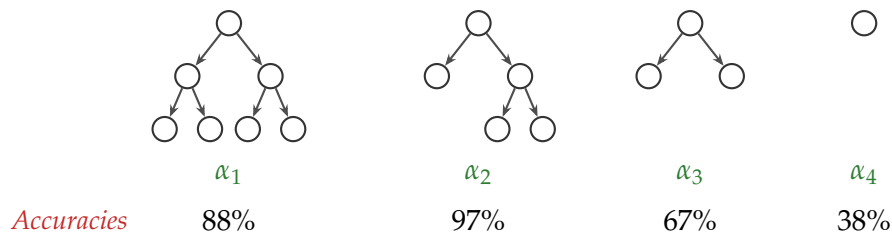
#Step 5:

The idea of K -fold cross validation is to train and test the tree on K different data.

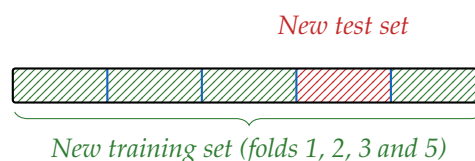
And this is the strategy: this is our first dataset:



- We train a tree on it and then get the accuracy on the testing data.
- Remember those alpha values we have found from the big tree. We apply them as a penalty, and then we retrain the tree on this same dataset.
- Since we have 4 alpha values, we have 4 trees on this dataset.
- We then compute the accuracy for each tree on the testing data:



- Now we re-split the data:



We repeat the algorithm then (trees, apply alpha, accuracy...), etc., until we have done all the combinations (each fold is the test set exactly once).

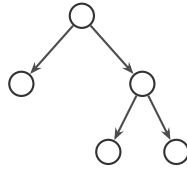
#Step 6:

For each tree with its associated alpha, we average the accuracy over the K folds, and we see which alpha-tree did the best job. Let's say it's α_2 .

#Step 7:

Remember Step 3, when we got multiple alphas associated with the trimmed big trees. Since α_2 did the best job on average,⁴ we choose the tree from Step 3 that's related to it:

⁴Editor's note. The notes say " α_1 " here, but Step 6 (and the accuracies of Step 5) selected α_2 .



Congratulations! We have pruned our tree, and we are (for now) safe (not completely) from overfitting.

That's all for Cost Complexity Pruning.

Random Forest

WE have seen that decision trees are pretty strong and fit the data well. The problem is that they fit the data *too much*. Even with pruning, decision trees suffer from *high variance*; that's why we introduce the *Random Forest*.

For this part especially, we will present the algorithm, then we will explain the math behind it.

Algorithm:

#Step 1: Bootstrapped dataset

Age	Weight	Do sport	Heart disease
72	90	No	Yes
52	72	No	No
15	70	Yes	No
61	77	No	Yes

The idea behind "Random Forest" is that we try to train multiple trees on slightly different data.

The idea of bootstrapping a dataset: choosing samples randomly from the dataset until making a new one that has the same size as the original.

Note that you can reuse a sample multiple times.

New bootstrapped dataset from the original one:

Age	Weight	Do sport	Heart disease
72	90	No	Yes
52	72	No	No
15	70	Yes	No
15	70	Yes	No

In this case, the sample

61	77	No	Yes
----	----	----	-----

 isn't in the bootstrapped dataset. It's called an "Out-of-Bag" sample. Keep that in mind.

Very Important Math

The probability of a sample to be chosen (in one draw) from the original dataset is $\frac{1}{n}$, with n the number of samples. So the probability of not being chosen is $1 - \frac{1}{n}$.

By computing the probability after each draw (we make n independent draws), the probability of never being chosen is:

$$\left(1 - \frac{1}{n}\right)^n$$

As n grows:

Euler limit

$$\lim_{n \rightarrow +\infty} \left(1 - \frac{1}{n}\right)^n = e^{-1} \approx 36.8\%^1 \quad \text{nearly } \frac{1}{3}$$

It means that the bootstrapped data contains nearly $\frac{2}{3}$ of the original data. The remaining $\frac{1}{3}$ goes to the Out-of-Bag dataset.

#Step 2: Randomly selecting features

Age	Weight	Do sport	Heart disease
72	90	No	Yes
52	72	No	No
15	70	Yes	No
15	70	Yes	No

This is our new dataset. Instead of using the initial method of trying the best split for each feature, we randomly choose $\lfloor \sqrt{p} \rfloor$ features, with p the total number of features.² (Rule of thumb, no math behind it: proposed by Breiman, and it worked empirically.)

Then we choose the best split between the selected features:

Age	Weight	Do sport	Heart disease
72	90	No	Yes
52	72	No	No
15	70	Yes	No
15	70	Yes	No

selected

Best split: Sport

Then we restart Step 2 (at every new node) until a stoppage condition is verified. And we have made the first tree of our forest.

Typically, the number of trees is 100, so we repeat from Step 1 until the desired number of trees is reached.

#Step 3: Bagging

Now let's say we have repeated the algorithm 100 times, and now we have 100 trees.

¹Editor's note. The notes say $\approx 38.2\%$; $e^{-1} = 0.3679\dots$

²Editor's note. The notes say " $\lfloor \sqrt{p} \rfloor$ samples, with p the total number of samples"; it is *features*. Note also that with $p = 3$ features $\lfloor \sqrt{3} \rfloor = 1$: two features are selected below only to make the illustration richer. For regression forests, Breiman's rule of thumb is $p/3$.

How are we going to make a decision?

We aggregate! Let's say we want to predict this:

Age	Weight	Do sport	Heart disease
57	80	No	??

We use each tree from the forest to make a prediction:

Trees	Prediction	The aggregate is:	
Tree 1	Yes	Yes	No
Tree 2	Yes		
⋮	⋮	82	18
Tree 100	No		

So the answer is "Yes". This process is called "*Bagging*": "**B**ootstrap **a**ggregating".

IV.1 *Advantages of Random Forest*

The advantage of random forest is that it reduces variance by adding randomness. But another thing is that it can also estimate its accuracy by using the Out-of-Bag dataset.

Process of estimating the forest accuracy:

We take each sample from the Out-of-Bag dataset, and we test it on all the trees that haven't been trained on that sample. We then see how much the forest is correct, and that's the accuracy (the *out-of-bag score*).

IV.2 *The Math Behind Random Forest*

In January 2001, Leo Breiman³ published for the first time the Random Forest algorithm. In the abstract, he said a very interesting sentence: the generalization error of a forest of tree classifiers "depends on the strength of the individual trees in the forest and the correlation between them".⁴

But what did he mean by this sentence??

We can roughly express the error in machine learning as follows:

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \underbrace{\text{Noise}}_{\text{from the data}}$$

- High bias and the model underfits.
- High variance and the model overfits.

³**Editor's note.** The notes write "Lea Breiman".

⁴L. Breiman, *Random Forests*, Machine Learning 45, 5–32 (2001), abstract.

So Breiman's sentence means: we need to balance between **bias** (strength of individual trees) and **variance** (correlation between them).

We know how to manage the strength of individual trees; how about variance?

Let's compute the variance for a single tree:

$$\text{Var}(T(x)) = \sigma^2 \quad (\sigma : \text{standard deviation})$$

Remark: $\text{Var}(T_1 + T_2) = \text{Var}(T_1) + \text{Var}(T_2) + 2 \text{Cov}(T_1, T_2)$

Utopic case: trees not related ($\text{Cov}(T_i, T_j) = 0 \quad \forall i \neq j$)

So, on average, the variance of n trees is:

$$\text{Var}\left(\frac{1}{n} \sum_{k=0}^{n-1} T_k(x)\right) = \frac{1}{n^2} n \sigma^2 = \frac{\sigma^2}{n}$$

This means: if we add more trees, the variance will decrease.

Reality:

We use the correlation coefficient ρ ($\rho = \text{Corr}(T_1, T_2)$):

$$\boxed{\text{Cov}(T_1, T_2) = \rho \sigma^2}$$

Demo:

$$\begin{aligned} \text{Cov}(T_i, T_j) &= \sqrt{\text{Var}(T_i) \text{Var}(T_j)} \times \text{Corr}(T_i, T_j) \\ &= \sqrt{\sigma^2 \sigma^2} \times \rho \\ &= \rho \sigma^2 \end{aligned}$$

Thus:

$$\begin{aligned} \text{Var}\left(\frac{1}{n} \sum_{k=0}^{n-1} T_k(x)\right) &= \frac{1}{n^2} \left(\sum_{k=0}^{n-1} \text{Var}(T_k(x)) + 2 \sum_{i < j} \text{Cov}(T_i, T_j) \right) \\ &= \frac{1}{n^2} \left(n \sigma^2 + 2 \sum_{i < j} \rho \sigma^2 \right) \end{aligned}$$

The number of possible tree pairs is $\binom{n}{2}$. So, for the forest F :

$$\begin{aligned}
 \text{Var}(F) &= \frac{1}{n^2} \left(n\sigma^2 + 2\binom{n}{2}\rho\sigma^2 \right) \\
 &= \frac{1}{n^2} \left(n\sigma^2 + 2\frac{n(n-1)}{2}\rho\sigma^2 \right) \\
 &= \frac{1}{n^2} \left(n\sigma^2 + n(n-1)\rho\sigma^2 \right) \\
 &= \sigma^2 \left(\frac{1}{n} + \frac{n-1}{n}\rho \right) \\
 &= \sigma^2 \left(\frac{1}{n} + \rho - \frac{\rho}{n} \right)
 \end{aligned}$$

“Golden formula”

$$\text{Var}(F) = \sigma^2 \left(\rho + \frac{1-\rho}{n} \right)$$

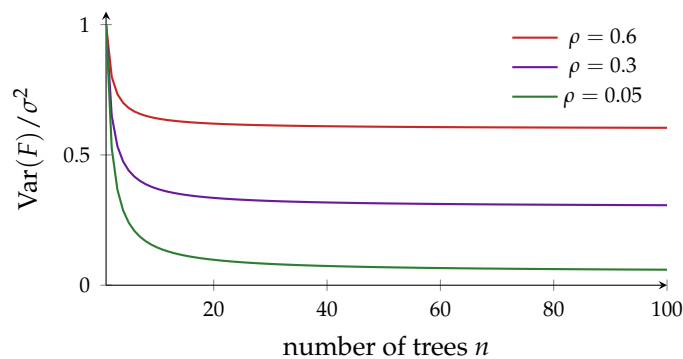
This formula **IS** Random Forest.

It means two things:

- If trees are correlated and we add more trees ($n \rightarrow +\infty$):

$$\text{Var}(F) \rightarrow \sigma^2\rho \quad (\text{no big improvements})$$

- But when we reduce the correlation ($\rho \rightarrow 0$), $\text{Var}(F)$ reduces drastically.



That’s why we randomly choose the bootstrapped dataset, and that’s why we randomly choose features.

“Because randomness reduces correlation.”

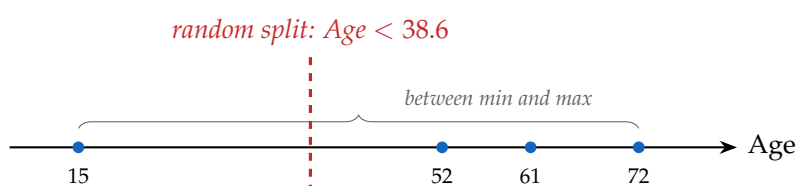
As the original Random Forest paper puts it (Section 3, *Using Random Features*), “the randomness injected has to minimize the correlation $\bar{\rho}$ while maintaining strength”.

Extra Trees

EVEN though it's another part, it's similar to random forest.¹ The only difference is: instead of using a bootstrapped dataset, we keep the same original dataset, and we still use the random feature selection.

But the twist is: when we randomly choose features, the best split for each feature is determined not by Gini impurity, but by complete randomness.

For example: we want to determine the best split for *age*. The min is 15 and the max is 72,² so we randomly select a number between 15 and 72.



Note: The Gini impurity is still used, to determine which randomly chosen split (one per selected feature) did a great job splitting the data.

Remark: Frameworks like scikit-learn still allow adding bootstrapping to Extra Trees. But, as the founders of this algorithm said, it's not necessary.

¹**Editor's note.** "Extra Trees" stands for *Extremely Randomized Trees*, introduced by P. Geurts, D. Ernst and L. Wehenkel in 2006.

²**Editor's note.** The notes say "the max is 77", which is the maximum *weight*; the ages of the dataset are 72, 52, 15 and 61.

CHAPTER
VI
AdaBoost

THIS part will serve as a bridge between the classical tree methods that we have seen before, and the “gradient boosting” based algorithms that we will see in the next parts.

AdaBoost is a method that relies on making multiple small trees, or “stumps”, called *weak learners*. The innovation here is that each tree has “a say”, which means an amount of importance. The other innovation is that each tree learns from the errors of the preceding tree.

That’s why it’s called “AdaBoost”:

- **Ada:** *Adaptive*, since each tree adapts to the errors of the preceding one.
- **Boosting:** by adding trees, the weak learners (or stumps) form together a strong classifier; it *boosts* the performance.

Algorithm:

#Step 1:

Let’s take this dataset:

Age	Weight	Height	Sick	Can join military	Sample weight
12	57	151	No	No	$\frac{1}{4}$
20	72	180	No	Yes	$\frac{1}{4}$
32	70	190	Yes	No	$\frac{1}{4}$
27	80	175	No	Yes	$\frac{1}{4}$

The sample weight will be used in the next step.

We first make a “stump”. A stump is a tree with only one node and two leaves:



#Step 2:

Since the tree is too small, its efficiency can be measured using the training data.

Let’s say that the first tree did a great job and got only one sample incorrect:

	Age	Weight	Height	Sick	Can join military	Sample weight
	12	57	151	No	No	$\frac{1}{4}$
	20	72	180	No	Yes	$\frac{1}{4}$
<i>incorrectly predicted →</i>	32	70	190	Yes	No	$\frac{1}{4}$
	27	80	175	No	Yes	$\frac{1}{4}$

As we said at the beginning, the novelty of the AdaBoost algorithm is that it introduces an “amount of say”, noted “ α ”, that will help us prioritize trees.

To compute it, we first compute the **error** ε :

$$\varepsilon = \sum_i w_i \quad \text{with } i \in \{\text{incorrectly predicted samples}\}$$

where w_i means the sample weight. Here $\varepsilon = \frac{1}{4}$.

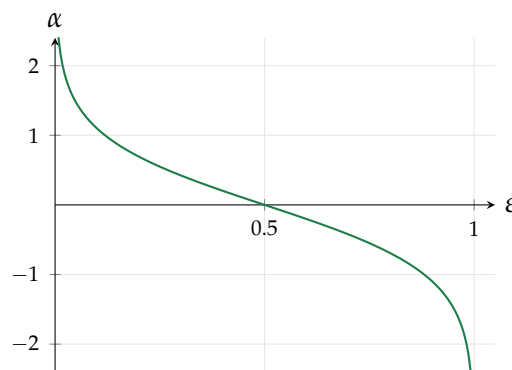
Then the “amount of say” α is:

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \varepsilon + \eta}{\varepsilon + \eta} \right)$$

Note: We add a tiny η (for example 10^{-10}) since ε may be equal to 1 (and $\ln(0)$ doesn’t exist), or equal to 0 (and we can’t divide by 0). We add η to avoid 0.¹

In our example: $\alpha = \frac{1}{2} \ln \left(\frac{1 - 1/4}{1/4} \right) = \frac{1}{2} \ln 3 \approx 0.549$.

This function didn’t come from nothing:



- If the error $\varepsilon \rightarrow 0$, the amount of say $\alpha \rightarrow +\infty$ (we give it a lot of importance).
- If the error $\varepsilon = 0.5$ (like flipping a coin), $\alpha = 0$ (we give absolutely no importance).
- If the error $\varepsilon \rightarrow 1$, the amount of say $\alpha \rightarrow -\infty$: the stump is almost always wrong, so its vote is reversed (it says “Yes”, we understand “No”).²

#Step 3: Redefining the sample weights

As I said at the beginning, the other novelty is that the next stumps learn from the errors of the previous ones. So the idea is: we give importance to the samples that have been incorrectly predicted.

¹**Editor’s note.** The notes wrote $\frac{1}{2} \log \left(\frac{1 - \varepsilon}{\varepsilon} + \eta \right)$. That protects against $\varepsilon = 1$ but not against the division by zero when $\varepsilon = 0$ (a perfect stump, which is very common). Adding η to both the numerator and the denominator protects both cases. The logarithm is the natural one.

²**Editor’s note.** The notes say that when $\varepsilon \rightarrow 1$ “we give absolutely no importance”. The point of *no* importance is $\varepsilon = 0.5$ ($\alpha = 0$); a negative α is a strong say in the *opposite* direction. The middle bullet was added to make this clear.

To do that, we use the “amount of say” α that we have already computed:

$$\begin{aligned} W_I &= w_i \times e^{\alpha} & W_I \text{ stands for the weight of an incorrectly predicted sample,} \\ W_c &= w_i \times e^{-\alpha} & W_c \text{ stands for the weight of a correctly predicted sample.} \end{aligned}$$

In our example: $W_I = \frac{1}{4}e^{0.549} \approx 0.433$ and $W_c = \frac{1}{4}e^{-0.549} \approx 0.144$.

We then **normalize** the weights by dividing by their sum ($0.433 + 3 \times 0.144 \approx 0.866$):³

Age	Weight	Height	Sick	Can join military	New sample weight
12	57	151	No	No	0.167
20	72	180	No	Yes	0.167
32	70	190	Yes	No	0.5
27	80	175	No	Yes	0.167

#Step 4:

We use these weights to make a new dataset.

The idea is: we choose a random number between 0 and 1, and we see in which interval the number is:

Age	Weight	Height	Sick	Can join military	New sample weight	Intervals
12	57	151	No	No	0.167	[0 ; 0.167]
20	72	180	No	Yes	0.167	]0.167 ; 0.333]
32	70	190	Yes	No	0.5	]0.333 ; 0.833]
27	80	175	No	Yes	0.167	]0.833 ; 1]

As we can see, the third interval is larger. Since the randomly chosen number is between 0 and 1, there are big chances that it falls in the third interval. The idea is to select the sample that corresponds to the interval where the random number fell in, and to repeat this until the new dataset has the same size as the original one.

Now the new dataset is:

	Age	Weight	Height	Sick	Military	Sample w.	
<i>Repeated 3 times</i>	32	70	190	Yes	No	$\frac{1}{4}$	<i>Sample weight initialized</i>
	20	72	180	No	Yes	$\frac{1}{4}$	
	32	70	190	Yes	No	$\frac{1}{4}$	
	32	70	190	Yes	No	$\frac{1}{4}$	

Now we can feel that the new dataset gives too much importance to the samples where the preceding stump failed.

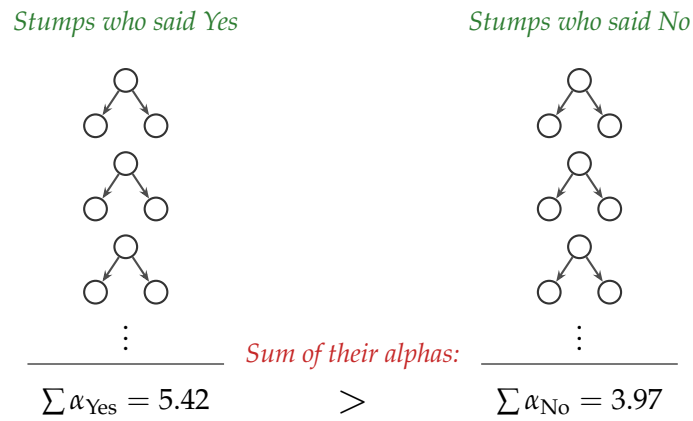
Then we repeat from Step 1 to make a new stump.

³**Editor's note.** The notes show the new weights 0.1, 0.1, 0.7, 0.1, which do not follow from the formulas. With $\varepsilon = \frac{1}{4}$ the computation gives $\frac{1}{6}, \frac{1}{6}, \frac{1}{2}, \frac{1}{6}$. This is not a coincidence: after the update, the misclassified samples always carry exactly half of the total weight, since $\varepsilon e^{\alpha} = (1 - \varepsilon)e^{-\alpha} = \sqrt{\varepsilon(1 - \varepsilon)}$.

VI.1 Making a Prediction

After making all the necessary stumps, how do we perform a prediction?

Let's say we have:



So the answer is Yes.

We give priority to the cluster of stumps that have the most amount of say.

Mathematically, with labels coded ± 1 and M stumps h_m , this is:

$$H(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m h_m(x) \right).$$

Gradient Boosting

GRADIENT Boosting is our last foundation algorithm in our series of decision trees. It's the foundation of the majority of the cutting-edge tree-based algorithms, like XGBoost, CatBoost, LightGBM...

Let's dive into it together.

Algorithm: GB for Regression:

The idea also relies on making trees that learn from the previous trees' errors, like AdaBoost, but in another way.

#Step 1:

Since we are studying gradient boosting for regression, the predicted value will be numerical.

The first tree we make will only be a *leaf* that contains the average value of all the values in the data.

Age	Does sport	Eats junk food	Weight
12	No	Yes	60
30	Yes	Yes	80
40	No	No	72
27	No	Yes	97

The first tree, as I said, is a leaf, and will contain the average of all the weights:

$$\boxed{\text{Weight} = 77.25} \quad \frac{60 + 80 + 72 + 97}{4} = 77.25$$

So the idea is: every sample is predicted to have a weight of 77.25 kg.

#Step 2:

As we have just seen, the first tree is too weak; we need to improve it!

The main idea is to compute the “*residuals*”:

$$\text{Res} = (\text{observed} - \text{predicted})$$

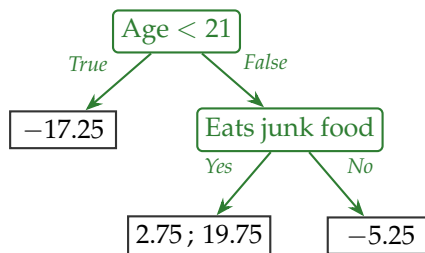
And the data becomes:

Age	Does sport	Eats junk food	Weight	Res
12	No	Yes	60	-17.25
30	Yes	Yes	80	2.75
40	No	No	72	-5.25
27	No	Yes	97	19.75

And plot twist! We don't make a new tree to predict the weight, but, this time, to predict the *residuals*. You will understand why in the next step.

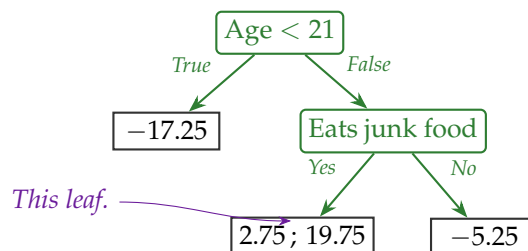
#Step 3:

We make a new tree from the residuals:¹

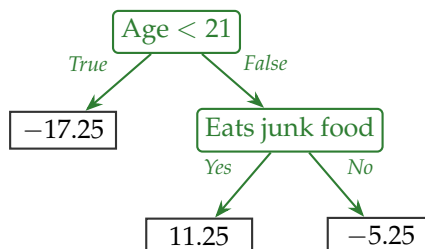


Note: The number of leaves is a stoppage condition for gradient boosting. Typically, it's between 8 and 32. It depends on the size of the data.

Now, let us look at the tree: there is a leaf that contains two values.

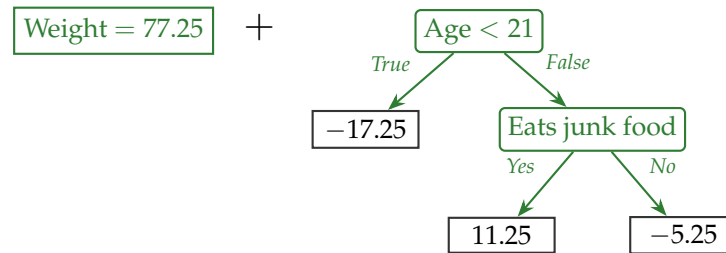


So we average the residuals: $\frac{2.75 + 19.75}{2} = 11.25$.



Congrats, we have made our first Gradient Boosting tree! To make a prediction, we just sum up the trees' predictions. That's why the trees predict the residuals: they play a *calibration* role.

¹**Editor's note.** The notes use "Does sport" for the second split, but with this data "Does sport = Yes" only contains the 30-year-old (2.75), while the leaf of the notes groups 2.75 and 19.75. The split that produces exactly the leaves of the notes is "Eats junk food" (Yes: 30 and 27 years old; No: 40 years old), so it was used here.



But is this correct??

Let's look at this sample and try to predict its value:

	Age	Does sport	Eats junk food	Weight	Res
	12	No	Yes	60	-17.25
	30	Yes	Yes	80	2.75
this row →	40	No	No	72	-5.25
	27	No	Yes	97	19.75

- The prediction of the first tree (the average tree) is "77.25".
- The prediction of the second tree (the residual tree) is "-5.25".

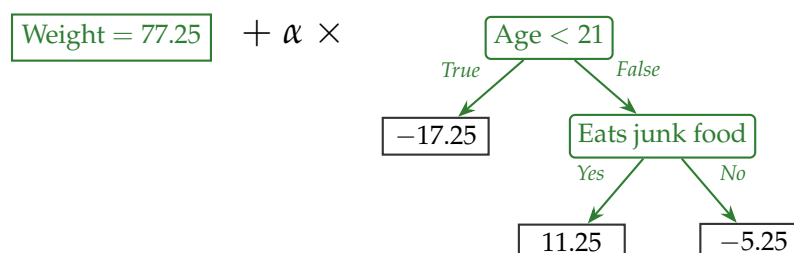
By summing the predictions up:

$$\text{predicted weight} = 77.25 - 5.25 = 72$$

Wow! It predicted it accurately!!

That's the big flaw!!!

That doesn't mean that the model is perfect and all good; it just means that the model fits the data too much (overfit). That's why, for Gradient Boosting, we add a **learning rate** α :



If we choose $\alpha = 0.1$, then the prediction would be:

$$\text{predicted weight} = 77.25 - 0.1 \times 5.25 = 76.725$$

Now we compute the residuals another time, each sample with the leaf it falls in:²

²**Editor's note.** In the notes, the new residuals were all computed with the prediction 76.725 of the 40-year-old (-16.725, 3.275, -4.725, 20.275). Each sample must use its *own* leaf: $77.25 - 0.1 \times 17.25 = 75.525$ for the 12-year-old, and $77.25 + 0.1 \times 11.25 = 78.375$ for the 30- and 27-year-olds. Notice that the corrected residuals still sum to 0, as they should with squared loss.

Age	Does sport	Eats junk food	Weight	Prediction	New Res
12	No	Yes	60	$77.25 - 0.1 \times 17.25 = 75.525$	-15.525
30	Yes	Yes	80	$77.25 + 0.1 \times 11.25 = 78.375$	1.625
40	No	No	72	$77.25 - 0.1 \times 5.25 = 76.725$	-4.725
27	No	Yes	97	$77.25 + 0.1 \times 11.25 = 78.375$	18.625

Every residual got a little bit smaller: we took a small step in the right direction. And we make a new tree to predict the new residuals.

We make new trees until the residuals are near 0, or the max number of trees is reached.

VII.1 Mathematical Details

Now that we have learned the main idea of gradient boosting, let's dive deeply into the math behind it.

We define the data as:

$$\{(x_i, y_i)\}_{i \in \{1, 2, \dots, n\}}$$

Since our goal is to minimize the error, we define a **loss function** that we are going to minimize. For one sample:

$$L(y_i, F(x_i)) = \frac{1}{2}(y_i - \hat{y}_i)^2, \quad \hat{y}_i = F(x_i),$$

and for the whole data we minimize $\sum_{i=1}^n L(y_i, F(x_i))$.³ The $\frac{1}{2}$ is only there to make the derivative cleaner.

#Step 1:

We try to make our first attempt at minimizing the loss, with a single constant value $\hat{y}$ for every sample ($F_0(x) = \hat{y}$). So we derive with respect to $F(x_i)$:

$$\begin{aligned}
 \frac{\partial}{\partial \hat{y}} \sum_{i=1}^n \frac{1}{2}(y_i - \hat{y})^2 &= - \sum_{i=1}^n (y_i - \hat{y}) = 0 \\
 \iff - \sum_{i=1}^n y_i + n\hat{y} &= 0 \\
 \iff n\hat{y} &= \sum_{i=1}^n y_i \\
 \iff \boxed{\hat{y} = \frac{1}{n} \sum_{i=1}^n y_i}
 \end{aligned}$$

"That's why the first guess is the average: it's our initial attempt."

³**Editor's note.** The notes define L directly as the sum $\frac{1}{2} \sum_{i=0}^{n-1} (y_i - \hat{y}_i)^2$. Separating the loss of *one* sample from the total makes Step 2 (a derivative per sample) consistent. Indices were also aligned to $1, \dots, n$.

In general (for any loss), Step 1 is written:

$$F_0(x) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, \gamma).$$

#Step 2:

For $m = 1, \dots, M$ (one tree at a time), we compute:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)}$$

sample
number
tree
number

The rest of this chapter completes the handwritten notes. The notes stop at the definition of r_{im} . What follows was written during the typesetting, in the spirit of the notes, to finish the algorithm, connect it with the example, and add the classification case.

Let's compute it for our loss:

$$\frac{\partial}{\partial F(x_i)} \frac{1}{2} (y_i - F(x_i))^2 = -(y_i - F(x_i)) \implies \boxed{r_{im} = y_i - F_{m-1}(x_i)}$$

Surprise! This is exactly the residual (observed – predicted) that we computed in the example. For another loss function, r_{im} is not exactly a residual anymore, but it plays the same role; that's why it's called a *pseudo-residual*.

*The trees don't really predict "residuals": they predict the negative gradient of the loss. That's why it's called **Gradient Boosting**.*

#Step 3:

We fit a regression tree (Chapter II) to the pseudo-residuals r_{im} , and we get its leaves, called *terminal regions*:

$$R_{jm}, \quad j = 1, \dots, J_m$$

where J_m is the number of leaves of the tree m (the "8 to 32" of the note above). In our example ($m = 1$), $J_1 = 3$:

$$R_{11} = \{\text{Age} < 21\}, \quad R_{21} = \{\text{Age} \geq 21, \text{ junk food}\}, \quad R_{31} = \{\text{Age} \geq 21, \text{ no junk food}\}.$$

#Step 4:

For each leaf, we compute the output value that minimizes the loss of the samples inside it:

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma)$$

With our squared loss:

$$\frac{\partial}{\partial \gamma} \sum_{x_i \in R_{jm}} \frac{1}{2} (y_i - F_{m-1}(x_i) - \gamma)^2 = - \sum_{x_i \in R_{jm}} (r_{im} - \gamma) = 0$$

$$\iff \boxed{\gamma_{jm} = \frac{1}{|R_{jm}|} \sum_{x_i \in R_{jm}} r_{im}}$$

That's why we averaged 2.75 and 19.75 to get 11.25! The leaf output is the average of the residuals that fall into it.

#Step 5:

We update the model:

$$F_m(x) = F_{m-1}(x) + \alpha \sum_{j=1}^{J_m} \gamma_{jm} \mathbb{1}(x \in R_{jm})$$

where α is the learning rate (in the literature it's often written ν or η , and called *shrinkage*); $\mathbb{1}(x \in R_{jm})$ is 1 if x falls in the leaf j , 0 otherwise. This is exactly “77.25 + 0.1 × tree”.

Then we go back to Step 2 with $m + 1$, until $m = M$.

The whole algorithm

Input: data $\{(x_i, y_i)\}_{i=1}^n$, a differentiable loss L , a number of trees M , a learning rate α .

1. $F_0(x) = \arg \min_{\gamma} \sum_i L(y_i, \gamma)$.
2. For $m = 1$ to M :
 - (a) $r_{im} = - \left[\frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)} \right]_{F=F_{m-1}}$ for $i = 1, \dots, n$;
 - (b) fit a regression tree to the $r_{im} \Rightarrow$ leaves $R_{jm}, j = 1, \dots, J_m$;
 - (c) $\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma)$;
 - (d) $F_m(x) = F_{m-1}(x) + \alpha \sum_j \gamma_{jm} \mathbb{1}(x \in R_{jm})$.
3. Output $F_M(x)$.

VII.2 Why “Gradient”?

Remember gradient descent: to minimize a function of parameters θ , we move against the gradient:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}(\theta).$$

In gradient boosting, the “parameters” are the predictions themselves, $(F(x_1), \dots, F(x_n))$. Moving them against the gradient gives:

$$F(x_i) \leftarrow F(x_i) - \alpha \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} = F(x_i) + \alpha r_i.$$

So why do we need a tree? Why not directly add αr_i to each prediction?

Because r_i only exists for the training samples! For a new person, we don't know y , so we can't compute his residual. The tree *learns* the negative gradient as a function of x , so it can *generalize* it to new data. Gradient boosting is gradient descent in the space of functions, where each step is a tree.⁴

Remark: Small learning rate means small steps: we need more trees, but each tree corrects only a little, and the model overfits much less. In practice, α is between 0.01 and 0.3, and we choose the number of trees M with cross validation (or early stopping on a validation set).

VII.3 Gradient Boosting for Classification

Now let's see the other side: predicting classes. Let's take this dataset (Yes = 1, No = 0):

Age	Weight	Do sport	Heart disease
72	90	No	Yes (1)
52	72	No	Yes (1)
15	70	Yes	No (0)
61	77	No	Yes (1)

The problem: we can't average "Yes" and "No", and a sum of trees can give any number (like -3.7 or 12) while a probability must be between 0 and 1. The trick is to make the trees predict the **log of the odds**, which lives on the whole real line, and to transform it into a probability only at the end:

$$F(x) = \ln\left(\frac{p}{1-p}\right) \iff p = \frac{e^{F(x)}}{1 + e^{F(x)}} = \frac{1}{1 + e^{-F(x)}}.$$

The loss function

For classification, we use the *negative log-likelihood* (also called log-loss, or binary cross-entropy):

$$L(y, p) = -[y \ln(p) + (1 - y) \ln(1 - p)].$$

Let's rewrite it with the log-odds F . Since $\ln(p) = F - \ln(1 + e^F)$ and $\ln(1 - p) = -\ln(1 + e^F)$:

$$\begin{aligned} L(y, F) &= -[y(F - \ln(1 + e^F)) - (1 - y) \ln(1 + e^F)] \\ &= \boxed{-yF + \ln(1 + e^F)} \end{aligned}$$

And its derivative is beautiful:

$$\frac{\partial L}{\partial F} = -y + \frac{e^F}{1 + e^F} = -y + p = -(y - p).$$

⁴J. H. Friedman, *Greedy Function Approximation: A Gradient Boosting Machine*, Annals of Statistics 29(5), 2001.

Step 1: the initial prediction

$$\frac{\partial}{\partial \gamma} \sum_{i=1}^n \left(-y_i \gamma + \ln(1 + e^\gamma) \right) = -\sum_i y_i + n p = 0 \iff p = \frac{1}{n} \sum_i y_i \iff F_0 = \ln \left(\frac{\# \text{Yes}}{\# \text{No}} \right).$$

The first leaf is the log of the odds of the data. In our example: $F_0 = \ln \frac{3}{1} \approx 1.099$, which means $p = 0.75$ for everyone.

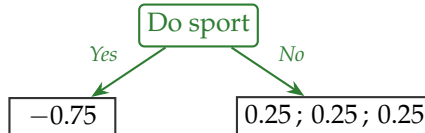
Step 2: the pseudo-residuals

$$r_{im} = -\frac{\partial L}{\partial F} = \boxed{y_i - p_i} \quad (\text{observed} - \text{predicted probability})$$

Again, a residual! In our example: $1 - 0.75 = 0.25$ for the three “Yes”, and $0 - 0.75 = -0.75$ for the “No”.

Step 3: a regression tree on the residuals

Let's say the best split is “Do sport”:



Step 4: the output of each leaf

Can we just average the residuals, like in regression?

No! The residuals are probabilities, but the tree must output a *log-odds* correction. We need to solve

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma),$$

and this time there is no simple closed form. The trick is a **second-order Taylor approximation** of the loss around $F_{m-1}(x_i)$:

$$L(y_i, F_{m-1} + \gamma) \approx L(y_i, F_{m-1}) + g_i \gamma + \frac{1}{2} h_i \gamma^2, \quad g_i = \frac{\partial L}{\partial F} = p_i - y_i, \quad h_i = \frac{\partial^2 L}{\partial F^2}.$$

Since $\frac{\partial p}{\partial F} = \frac{e^F}{(1 + e^F)^2} = p(1 - p)$, we get $h_i = p_i(1 - p_i)$. Now we derive the approximation with respect to γ :

$$\sum_{x_i \in R_{jm}} (g_i + h_i \gamma) = 0 \iff \gamma_{jm} = \frac{-\sum g_i}{\sum h_i} = \frac{\sum_{x_i \in R_{jm}} (y_i - p_i)}{\sum_{x_i \in R_{jm}} p_i(1 - p_i)}$$

*Sum of the residuals divided by the sum of $p(1 - p)$.
Keep this “gradient over hessian” idea in mind: XGBoost is built on it.*

In our example, $p_i(1 - p_i) = 0.75 \times 0.25 = 0.1875$ for everyone:

$$\gamma_{\text{Yes}} = \frac{-0.75}{0.1875} = -4, \quad \gamma_{\text{No}} = \frac{0.25 + 0.25 + 0.25}{3 \times 0.1875} = \frac{0.75}{0.5625} \approx 1.333.$$

Step 5: the update

With $\alpha = 0.1$:

- Does sport: $F_1 = 1.099 + 0.1 \times (-4) = 0.699$, so $p = \frac{1}{1 + e^{-0.699}} \approx 0.668$ (it was 0.75: it goes down, good, this person has no heart disease).
- Doesn't do sport: $F_1 = 1.099 + 0.1 \times 1.333 = 1.232$, so $p \approx 0.774$ (it was 0.75: it goes up, good).

The new residuals are $1 - 0.774 = 0.226$ for the three "Yes" and $0 - 0.668 = -0.668$ for the "No": all of them got smaller. Then we make a new tree, etc.

Making a prediction

We sum the log-odds of all the trees, and we transform the result into a probability:

$$p(x) = \frac{1}{1 + e^{-F_M(x)}}, \quad \text{answer} = \begin{cases} \text{Yes} & \text{if } p(x) > 0.5 \\ \text{No} & \text{otherwise.} \end{cases}$$

VII.4 Good to Know Info

- **Main hyperparameters:** the number of trees M , the learning rate α , the size of each tree (max number of leaves or max depth), and the minimum number of samples per leaf.
- **Stochastic gradient boosting:** Friedman (2002) proposed to train each tree on a random subsample of the data (without replacement), like the randomness of Random Forest. It reduces overfitting, and it's faster.
- **Other losses:** any differentiable loss works, only the pseudo-residuals and the leaf outputs change: absolute error (robust to outliers), Huber loss, softmax/multi-class log-loss (one tree per class at each round)...
- **AdaBoost is a relative:** AdaBoost can be derived as the same kind of stage-wise additive model, built with the *exponential* loss $L(y, F) = e^{-yF}$ (with $y \in \{-1, 1\}$) instead of the log-loss (Friedman, Hastie & Tibshirani, 2000).

Gradient boosting is the foundation. The next algorithms (XGBoost, CatBoost, LightGBM) keep the same idea and make it more precise, more regularized, and much faster.

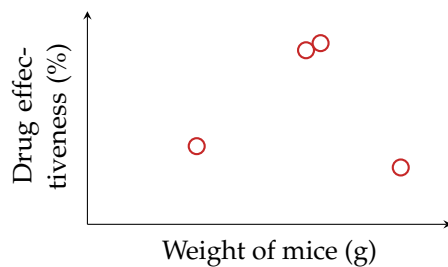
GBDT Based Algorithms: XGBoost

SINCE we have learned about Gradient Boosting Decision Trees (GBDT), many variants have appeared. These variants introduce some enhancements in terms of speed and/or accuracy. In this sequence of chapters, we will talk about some of them, and dive into how they work.

VIII.1 XGBoost

XGBoost stands for “**eXtreme Gradient Boosting**”. And these are the main ideas.

Let’s say we are in a regression problem:



So we want to predict the % of the drug effectiveness, knowing the weight of the mice, using XGBoost.

Like GBDT, assuming that we are using the MSE as the loss function, the first prediction is the average of all y_i :

$$\frac{22 + 49 + 51 + 16}{4} = \underbrace{34.5}_{\text{First tree}}$$

Weight	Effectiveness	Residuals ($y_i - \hat{y}_i$)
15	22	−12.5
30	49	14.5
32	51	16.5
43	16	−18.5

So, same thing as GBDT models: the next tree should predict the residuals, and learn from the preceding error. But the way trees are made in XGBoost is pretty unique.

Algorithm:

We collect all the residuals, and we compute the “**Similarity**”:

$$\text{Similarity} = \frac{(\sum \text{Residuals})^2}{\# \text{Residuals} + \lambda}$$

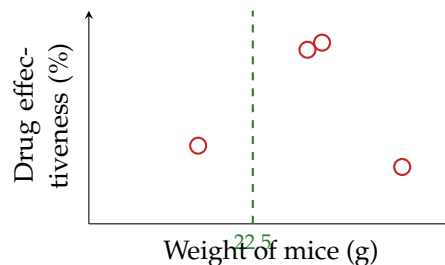
λ : regularization, we will talk about it later.¹

Then, for our case, the similarity of our root is:

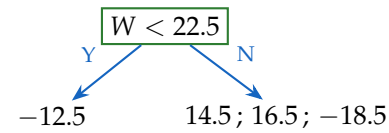
$$\text{Sim}_{\text{root}} = \frac{(-12.5 + 14.5 + 16.5 - 18.5)^2}{4 + 0} \quad (\text{we take } \lambda = 0 \text{ for now})$$

$$\text{Sim}_{\text{root}} = 0 \quad (\text{Well, by total chance, I didn't want it to be } 0)^2$$

Now, as for decision trees, we try to find the best split:



The first split would be 22.5 (the average of 15 and 30):



We then compute the similarity of each leaf:

$$\text{Sim}_{\text{Right}} = \frac{(14.5 + 16.5 - 18.5)^2}{3} = \frac{12.5^2}{3} \approx 52.08$$

$$\text{Sim}_{\text{Left}} = \frac{(-12.5)^2}{1} = 156.25$$

Then, to evaluate how good the split is, we compute the **gain**:

¹**Editor's note.** Where does this formula come from? It is the “gradient over hessian” idea of Chapter VII, plus a penalty $\frac{1}{2}\lambda w^2$ on the output w of each leaf. For a leaf, XGBoost minimizes $\sum_i (g_i w + \frac{1}{2} h_i w^2) + \frac{1}{2} \lambda w^2$, whose minimum is reached at $w^* = -\frac{G}{H + \lambda}$ and is equal to $-\frac{1}{2} \frac{G^2}{H + \lambda}$, with $G = \sum g_i$ and $H = \sum h_i$. With the squared loss, $g_i = -(\text{residual}_i)$ and $h_i = 1$, so $H = \# \text{Residuals}$: the Similarity is exactly $\frac{G^2}{H + \lambda}$ (the official gain also carries a factor $\frac{1}{2}$, which does not change which split is the best), and the output value given below is w^* . This is also the formula of Chapter X.

²**Editor's note.** In fact it is not a chance: with the MSE, the first prediction is the average, and the residuals around an average always sum to 0. So the similarity of the root of the *first* tree is always 0 when $\lambda = 0$.

$$\text{Gain}_{\text{split}} = \text{Sim}_{\text{Right}} + \text{Sim}_{\text{Left}} - \text{Sim}_{\text{Root}} - \gamma$$

γ is a number that helps XGBoost decide which tree to prune.

As a rule, if the gain of a subtree is negative, we prune the subtree. As you may have thought, γ could affect the sign of the gain, which helps us manage the aggressiveness of the pruning.

⚠ Important remark!

λ and γ are linked. Let's see it in this simulation, with $\gamma = 100$.

The gain of the tree that we have made earlier is:

$$\text{Gain}_{\text{tree}} = 156.25 + 52.08 - 0 - 100 = 108.33 > 0$$

So we keep the tree. But remember, we took $\lambda = 0$.

If $\lambda > 0$, for example $\lambda = 2$:

$$\text{Sim}_{\text{Right}} = \frac{(14.5 + 16.5 - 18.5)^2}{3 + 2} = 31.25$$

$$\text{Sim}_{\text{Left}} = \frac{(-12.5)^2}{1 + 2} \approx 52.08$$

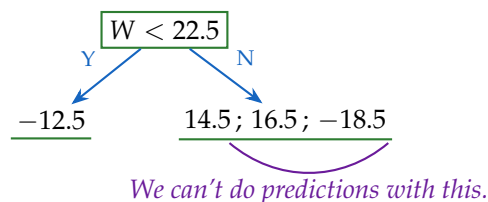
$$\text{Gain}_{\text{tree}} = 31.25 + 52.08 - 0 - 100 = 83.33 - 100 = -16.67 < 0$$

We prune the tree.

As we can see, by increasing the value of λ and keeping the same γ , we pruned the tree (or subtree). Pruning helps avoid overfitting, so λ helps us control how much the model adapts to the data.

That's why λ is meant for regularization.

Then we repeat for each split. Let's say the best split is $W < 22.5$, with these parameters: $\lambda = 0$ and $\gamma = 100$. So this is our first tree:



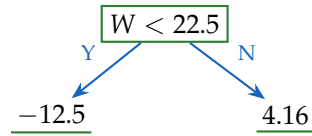
So, to make the prediction, there is a formula:

$$\text{Pred} = \frac{\sum \text{Residuals}}{\# \text{Residuals} + \lambda}$$

For our case:

$$\text{Pred}_R = \frac{14.5 + 16.5 - 18.5}{3} \approx 4.16, \quad \text{Pred}_L = -12.5$$

Then the tree becomes:



And, until now, our XGBoost model is:

$$\boxed{34.5} + \eta \times \begin{array}{c} \boxed{W < 22.5} \\ \swarrow \text{Y} \quad \searrow \text{N} \\ \underline{-12.5} \quad \underline{4.16} \end{array}$$

Remember, η is the learning rate. And then we repeat.

Remark: These are the main ideas of how XGBoost works. It also has some other tricks to quickly find the best split, using quantiles.

GBDT Based Algorithms: CatBoost

CATBOOST stands for **C**ategorical **B**oosting (not cat, the pet). Its idea comes from treating categorical values in a special, *sequential* way.

IX.1 Target Encoding

Before we talk about CatBoost, we should talk about *target encoding*.

Age	Eyes color	Likes Superman
12	Blue	0
22	Blue	1
37	Brown	1
17	Green	0
25	Brown	1

As we have seen in this book, decision trees treat categorical variables very well.

But linear models find difficulties with them. Then comes **target encoding**, with this formula:

$$\text{Code}_n = \frac{\sum_{i: x_i=n} y_i}{\#\{i : x_i = n\}}$$

i.e. the category n is replaced by the average of the targets of the samples of this category.

Applying it to our dataset:

- Blue: $\frac{1+0}{2} = 0.5$
 - Brown: $\frac{1+1}{2} = 1$
 - Green: 0
- All categorical variables are now encoded, as simple as that.*



(Maybe the biggest warning in this book)
“Data Leakage Warning”

The encoding idea is pretty good, but the model is trained on features that contain information about the target *by construction*. This is “pure” leakage.

That’s the problem CatBoost wants to solve, by introducing:

“Ordered Target Encoding”

IX.2 Ordered Target Encoding

Well, the main idea of ordered target encoding is as follows:

Age	Eyes color	Likes Superman
12	Blue	0
22	Blue	1
37	Brown	1
17	Green	0
25	Brown	1

← We assume, at the beginning, that the dataset has only this one sample.

Since we have a categorical column, we should encode it with this formula:

$$\text{Code}_n = \frac{\sum y_i | x_i = n + \lambda \beta}{\# y_i + \lambda}$$

The sum of the preceding targets of the class (not the actual one) *Initial default number (prior)* *Smoothing parameter, used to avoid division by 0, and also as a weight for β*

To encode *Blue* (first row), with $\lambda = 1$, we do:

$$\text{Code}_{\text{Blue}} = \frac{\overbrace{0}^{\text{since no preceding targets}} + 1 \times 0.6}{0 + \underbrace{1}_{\text{avoid division by 0}}} = 0.6 \quad (\text{we choose } \beta = 0.6, \text{ but we will talk about it later})$$

Then we add the second row to the “known” data, and we encode it using only the rows before it, and so on.¹

¹**Editor’s note.** The notes jump directly to the encoded column. The intermediate computations were added to show where each value comes from.

Age	Eyes color	Likes Superman	Encoded eyes color
12	Blue	0	$\frac{0+0.6}{0+1} = 0.6$
22	Blue	1	$\frac{0+0.6}{1+1} = 0.3$ (one preceding Blue, with target 0)
37	Brown	1	$\frac{0+0.6}{0+1} = 0.6$
17	Green	0	$\frac{0+0.6}{0+1} = 0.6$
25	Brown	1	$\frac{1+0.6}{1+1} = 0.8$ (one preceding Brown, with target 1)

About β ?

Well, some resources take 0.05 as a default value for β , and it's okay! But in reality, β is computed, for either regression or classification, as the mean of all the target values (whether they are associated with the class or not):

$$\beta = \frac{0 + 1 + 1 + 0 + 1}{5} = \frac{3}{5} = 0.6$$

IX.3 The Training

Then comes the training part. As for all GBDT models, we start at the beginning with an initial prediction. For CatBoost, the initial prediction is 0.²

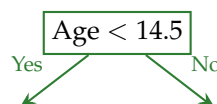
First tree 0

Then we compute the residuals (this is the dataset with all the categories encoded):

Age	Eyes color	Residuals
12	0.6	0
22	0.3	1
37	0.6	1
17	0.6	0
25	0.8	1

Then we make another tree to predict the residuals. CatBoost trees are very unique.

We first select the threshold that we want to split the data with. Let's say $\text{Age} < 14.5$:



Then we fill the leaves and the predictions *sequentially*: the prediction of a row is the average of the *preceding* residuals that fell in the same leaf; at the beginning, it's 0.

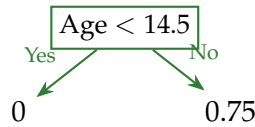
²**Editor's note.** This is the classic behaviour (and still the default for classification, where 0 is a log-odds, i.e. a probability of 0.5). For regression, recent versions of CatBoost start from the average of the target, like Chapter VII.

Age	Eyes color	Likes Superman	Leaf	Prediction
12	0.6	0	Yes	0 (nothing before it in "Yes")
22	0.3	1	No	0 (nothing before it in "No")
37	0.6	1	No	1 (average of: 1)
17	0.6	0	No	1 (average of: 1, 1)
25	0.8	1	No	0.66 (average of: 1, 1, 0)

Then, as for all GBDT models, the model after n trees is:

$$F_n(x) = F_{n-1}(x) + \eta T_n(x)^3$$

And the tree becomes:⁴



How good is the split?

As for all decision tree algorithms, we need to measure how good the split is. For CatBoost, we use the **Cosine Similarity** between *Likes Superman* and the *Predictions*:⁵

$$\text{Cosine Similarity} = \frac{y \cdot \hat{y}}{\|y\|_2 \|\hat{y}\|_2}$$

Or, analytically:

$$\text{Cosine Similarity} = \frac{\sum_{i=1}^N y_i \hat{y}_i}{\left(\sqrt{\sum_{i=1}^N y_i^2} \right) \left(\sqrt{\sum_{i=1}^N \hat{y}_i^2} \right)}$$

with N the number of rows. In our example:

$$\frac{0 + 0 + 1 + 0 + 0.66}{\sqrt{3} \sqrt{0 + 0 + 1 + 1 + 0.66^2}} = \frac{1.66}{1.732 \times 1.563} \approx 0.62$$

The best split is the one with the highest cosine similarity.

³**Editor's note.** The notes write $\text{Pred}_n = T_{n-1} + \eta T_n$; the previous term is the whole model built so far (F_{n-1} , the sum of all the previous trees), not only the previous tree.

⁴**Editor's note.** The notes put 0.66 in the right leaf, which is the *sequential* prediction of the last row. The sequential (ordered) predictions are only used during training, to judge the split without leakage. Once the tree is chosen, the value stored in a leaf, used to predict new data, is computed with all the residuals of the leaf: $\frac{1+1+0+1}{4} = 0.75$.

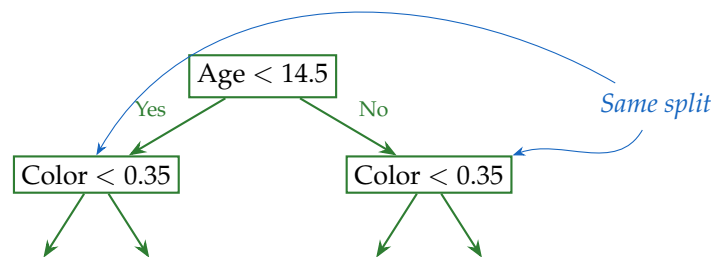
⁵**Editor's note.** More generally, CatBoost compares the residuals (the gradients) with the leaf predictions. For this first tree, the residuals are exactly the targets (the initial prediction is 0), so it's the same thing.

IX.4 Oblivious Trees

⚠ Important Remark:

Choosing which split does a great job using cosine similarities could be pretty slow, especially for large datasets.

That's why CatBoost introduces **symmetric trees**, or “**oblivious**” trees. Oblivious trees are trees whose splits are the same between left and right (at the same depth, every node asks the same question):



The main cause is, first, to make the trees *weaker*: like for AdaBoost, a succession of weak learners that learn from the preceding errors becomes stronger.

The second reason is, as I said earlier, that finding the best split and computing the cosine similarity could take too much time. That's why we repeat the same split in all the branches: the algorithm won't bother about which direction to use.

GBDT Based Algorithms: LightGBM

As I said in the beginning, GBDT based models exist to improve GBDT in either speed or accuracy. *LightGBM* aims to improve speed, by progressively reducing the dataset size, the features, and also the splits.

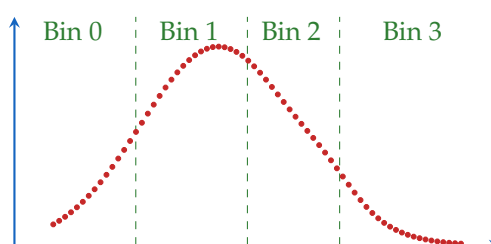
That's why it's called LightGBM, which stands for **Light Gradient Boosting Machine**: it makes Gradient Boosting "lighter".

It comes with 4 improvements.

X.1 Histograms

Let's say we have a dataset with multiple columns, and some of them represent a continuous distribution of a variable. If we try to make a tree, we would need a threshold; if the dataset has 10^6 rows, that would take a lot of time (there would be up to 10^6 candidate thresholds for each feature).

LightGBM came up with an idea: instead of searching for the best threshold in the whole dataset, we convert the continuous variable columns into "**bins**".¹



So the column becomes:

Age		Age bins
72		3
16	→	0
29		1
57		3
33		2

*Harder to find
a threshold*

*Easier to find
a threshold*

¹**Editor's note.** In the notes, the drawing numbers the bins from 1 to 4 while the table uses 0 to 3; the drawing was renumbered from 0.

X.2 EFB

EFB stands for **Exclusive Feature Bundling**. The idea is very easy to understand:

“We regroup the features that are mutually exclusive.”

As an example:

Male	Female
1	0
0	1
1	0
1	0
0	1

If a dataset contains the columns *Female* and *Male*, these two columns are mutually exclusive. This means: if one is not 0, the other is 0 (they are never non-zero at the same time).²

LightGBM regroups the mutually exclusive columns into one:

Gender
1
2
1
1
2

Note: You may think that it would be better if we assign 1 to male and 0 to female, but that’s not how EFB works.

To group the columns, EFB would allow an interval for each column: for male it’s $\{1\}$, and for female it’s $\{2\}$ (0 stays for “none of them”).³ We didn’t see the intervals clearly in this example, but let’s take some high-cardinality features:

Feature A	Feature B	Bundle
0	7	$7 + 5 = 12$
2	0	2
5	0	5
0	6	$6 + 5 = 11$
$\vdots$	$\vdots$	$\vdots$

In this case, we assign the interval $[0, 5]$ to feature A, and $[6, \dots]$ to feature B: the values of B are shifted by 5 (the maximum of A), so a value of the bundle tells us exactly which feature it comes from.

²**Editor’s note.** The notes say “if one is 0 the other isn’t 0”. Mutual exclusivity means the two features are never non-zero *together*; both can be 0 at the same time (for example when the gender is unknown).

³**Editor’s note.** The notes write the bundled values as “11” (male) and “10” (female). EFB keeps the values of the first feature and *shifts* the non-zero values of the next one by an offset (here +1), so the bundle is 1 for male and 2 for female, and 0 means both columns are 0.

x.3 GOSS

GOSS is one of the major improvements brought by LightGBM. GOSS stands for **G**radient-**B**ased **O**ne-**S**ide **S**ampling, and these are the main ideas.

The gradient helps us know which direction we should take to reach the optimum. But indirectly, it could also tell us whether, until now, we are doing a great job or not.

Even better, it could also tell us which sample in our dataset struggles the most (a big gradient means a big error). So LightGBM came up with this question:

If we know which samples my model struggles to learn, why not just train it on those samples?

And the answer is GOSS!

1/ First, we compute the gradient for each sample:

Rows	Gradient
1	22.7
2	3.8
3	53.2
4	13.6
⋮	⋮

2/ Second, we sort them (by the size of the gradient):

Rows	Gradient
3	53.2
1	22.7
4	13.6
2	3.8
⋮	⋮

3/ We take only the 20% highest gradients, and from the remaining 80% we randomly choose 10% of them, to make a new dataset:

Rows	Gradient	
3	53.2	<i>20% goes to the new dataset</i>
1	22.7	
4	13.6	<i>10% are chosen randomly from the remaining 80%</i>
2	3.8	
⋮	⋮	

That's why it's called *Gradient-Based One-Side Sampling*: the sampling is chosen using the gradient, and we only sample on one side (the side of the small gradients).

For example, if we had a dataset with 10^6 rows, we would use:

- $20\% \times 10^6 = 200\,000$ rows (the highest gradients);
- from the remaining 800 000, we choose 80 000 (10%) randomly.

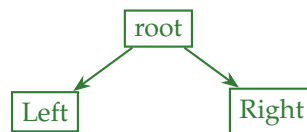
So the new dataset that we are going to train our new trees on has:

280 000 rows, instead of 1 000 000 rows. **Huge reduction.**⁴

X.4 Leaf-Wise Growth

Each GBDT algorithm has its own tree structure, like what we have seen with XGBoost (which grows its trees level by level) and the CatBoost oblivious trees. LightGBM introduces a **leaf-wise** growth, which means we split only the leaf that will give us the most gain (information).

Let's say we have this tree:

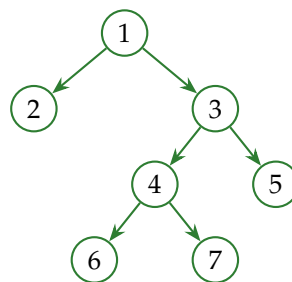


Now we should make a decision: either split *Left* or *Right*. For this reason, we have this formula:⁵

$$\text{Gain} = \left(\frac{G_R^2}{H_R + \lambda} + \frac{G_L^2}{H_L + \lambda} - \frac{G^2}{H + \lambda} - \gamma \right)$$

- G : sum of gradients (of the node; G_L , G_R for the left and right children);
- H : sum of Hessians;
- λ : L_2 regularization;
- γ : minimum gain required to make a split.

Therefore, we can measure how good a split is, and which split to use. We could finish with a tree like this:



⁴**Editor's note.** Two details of the real algorithm. (1) In LightGBM (and in the GOSS paper) the "10%" is a fraction of the *whole* dataset (`top_rate` = $a = 0.2$, `other_rate` = $b = 0.1$), which would give $200\,000 + 100\,000 = 300\,000$ rows. (2) The randomly chosen small-gradient samples are multiplied by a weight $\frac{1-a}{b}$ ($= 8$ here) when the gains are computed, so that they still represent all the small-gradient samples that were dropped, and the data distribution is not biased.

⁵**Editor's note.** The papers of XGBoost and LightGBM put a factor $\frac{1}{2}$ in front of the three similarity terms: $\text{Gain} = \frac{1}{2}(\dots) - \gamma$. It comes from the second-order approximation of Chapter VII. Without it, the gain is simply doubled; it doesn't change which split is the best, only the scale of γ . The notes' convention (the same as in the XGBoost chapter) was kept.

Notes on this Edition

This book is a faithful transcription of the author's handwritten notes. The wording, the examples and the order of the explanations are the author's. The typesetting only corrected grammar and spelling silently; everything that changes a number, a formula, a name or a date is listed below (and footnoted in the text as **Editor's note**).

Chapter I: Classification Trees

- CART was published in 1984 (the notes say 1980).
- The Gini impurity is *named after* Corrado Gini (he introduced the Gini coefficient); the notes say "introduced by".
- Gini and entropy lead to the same split "almost always", not always.
- The demonstration $P(A \cap A) = P(A)^2 \iff P(A) \in \{0,1\}$ now states that " A is independent of itself" is the hypothesis, not a general fact.
- p_i^2 is explained with two *independent* draws (first property), instead of $P(i \cap i)$ (which equals p_i for a single event).
- The "impurity must decrease" remark was made precise: the weighted impurity of the children can never exceed the impurity of the parent (with the numbers of the example).
- Information gain uses the *weighted* entropies of the children.

Chapter II: Regression Trees

- The SSR is computed on the target y_i , not on X_i .
- The split threshold is the average of two neighbouring *feature* values.
- The split "< 2 days" was harmonised with the axis (hours of training). The SSR curve is plotted against the candidate thresholds, and was computed from the plotted data.

Chapter III: Cost Complexity Pruning

- What the notes call "Leave One Out Cross Validation (LOOCV)" is K -fold cross validation (LOOCV is the case $K = n$).
- Step 7 selects α_2 (the notes say α_1 , in contradiction with Step 6 and the accuracies).
- Added: a hold-out test set should stay outside of pruning; the weakest-link formula for the next α .

Chapter IV: Random Forest

- $e^{-1} \approx 36.8\%$ (the notes say 38.2%).
- Random Forest samples $\lfloor \sqrt{p} \rfloor$ features (p = number of features), not samples.
- “Lea Breiman” $\rightarrow$ Leo Breiman. The highlighted sentences of the original paper are quoted in part, with a reference.
- Added: a plot of the golden formula for three values of ρ .

Chapter V: Extra Trees

- The maximum age of the dataset is 72 (the notes say 77, the maximum weight).

Chapter VI: AdaBoost

- The amount of say is $\frac{1}{2} \ln \frac{1-\varepsilon+\eta}{\varepsilon+\eta}$, protected against both $\varepsilon = 0$ and $\varepsilon = 1$.
- When $\varepsilon \rightarrow 1$, $\alpha \rightarrow -\infty$ means the vote is reversed; “no importance” is $\varepsilon = 0.5$.
- The new sample weights are $\frac{1}{6}, \frac{1}{6}, \frac{1}{2}, \frac{1}{6}$ (the notes show 0.1, 0.1, 0.7, 0.1), and the intervals were updated accordingly.

Chapter VII: Gradient Boosting

- The second split of the residual tree is “Eats junk food” (“Does sport” does not give the leaves of the notes).
- The new residuals after the learning rate are $-15.525, 1.625, -4.725, 18.625$ (each sample uses its own leaf).
- The loss is defined per sample, and its total is minimized; indices aligned to $1, \dots, n$; missing minus sign in the derivative of Step 1.
- **Completed during the typesetting** (the notes stop at the definition of r_{im}): the computation of the pseudo-residuals, Steps 3 to 5, the whole algorithm, “Why gradient?”, Gradient Boosting for classification with its worked example, and the “Good to know” section.

Chapter VIII: XGBoost

- The similarity of the root is 0 not by chance: residuals around the average always sum to 0.
- Added (footnote): where the Similarity and the output value come from $(\frac{G^2}{H+\lambda})$ and $-\frac{G}{H+\lambda}$, and the factor $\frac{1}{2}$ of the official gain.

Chapter IX: CatBoost

- The intermediate computations of the ordered target encoding were added (the notes give only the encoded column); the cosine similarity of the example was computed (≈ 0.62).
- The model after n trees is $F_n = F_{n-1} + \eta T_n$ (the notes write $T_{n-1} + \eta T_n$).

- The stored value of the right leaf is 0.75 (all its residuals); 0.66 is the ordered prediction of the last row, used only to score the split.
- Precisions: the initial prediction 0 is the classic/classification behaviour; the cosine similarity compares residuals with predictions.

Chapter X: LightGBM

- The bins of the drawing were renumbered 0 to 3, to match the table.
- Mutually exclusive features are never non-zero together (both can be 0).
- EFB bundles Male/Female as 1 and 2 (the notes write 11 and 10), by shifting the second feature with an offset.
- GOSS: in LightGBM the 10% is a fraction of the whole dataset, and the sampled small-gradient rows are re-weighted by $\frac{1-a}{b}$.
- The gain of the papers has a factor $\frac{1}{2}$; the notes' convention was kept (it only rescales γ).

*From one question asked to the data
to the algorithms that win competitions.*

Why is a decision tree built with the Gini impurity, and where does that formula come from? Why does a forest of trees generalize better than one tree, and what does Breiman's "golden formula" really say? How can a tree learn from the errors of another tree, and why is it called *gradient* boosting?

The Enchiridion for Tree Based Algorithms Theory answers these questions one step at a time, with small worked examples computed by hand, and with every demonstration written down, from Classification and Regression Trees, Cost Complexity Pruning, Random Forest and Extra Trees, to AdaBoost, Gradient Boosting, XGBoost, CatBoost and LightGBM.

Written by hand by Reda Louriki, and typeset in L^AT_EX by Claude Opus 5.5.

REDA LOURIKI

